N048 GitHub Actions로 자동 배포하기_해설서

Chapter01

/강의정리 챕터 01. GitHub Actions로 자동 배포하기

1. 학습 목적

이번 강의의 핵심은 코드를 수정할 때마다 사람이 직접 배포 명령을 실행하지 않아도, GitHub가 정해진 절차에 따라 서비스를 자동으로 배포하도록 만드는 것입니다.

지금까지는 AI 친구 에이전트 서비스를 FastAPI로 만들고 Google Cloud Run에 배포했다면, 이제는 그 서비스의 개발부터 배포까지 이어지는 과정을 자동화합니다.

중학생도 이해할 수 있도록 비유하면, 지금까지는 음식을 만들 때마다 직접 포장하고 배달까지 했다면, 이제는 음식을 완성해 정해진 장소에 올려놓으면 포장과 배달이 자동으로 진행되는 시스템을 만드는 것입니다.

2. 중학생도 이해하는 핵심 개념

① CI와 CD란 무엇인가요?

CI (지속적 통합, Continuous Integration)

개발자가 코드를 변경하면 자동으로 코드를 가져와 검사하고 테스트하는 과정입니다.

비유: 제품을 출고하기 전에 품질 검사를 하는 것

CD (지속적 제공·배포, Continuous Delivery/Deployment)

검사된 코드를 배포할 준비를 하거나 실제 서비스에 자동으로 반영하는 과정입니다.

비유: 품질 검사를 통과한 제품을 매장에 진열하는 것

CD는 두 가지 의미로 쓰일 수 있습니다.

- Continuous Delivery(지속적 제공): 배포할 준비까지 자동화하고, 실제 배포는 사람이 승인할 수 있습니다.

- Continuous Deployment(지속적 배포): 검사를 통과한 변경 사항을 실제 서비스에 자동으로 배포합니다.

이번 강의에서는 GitHub Actions를 이용해 Google Cloud Run에 자동 배포하는 흐름을 다룹니다.

② GitHub Actions란 무엇인가요?

GitHub 저장소에서 특정 사건이 발생했을 때 자동으로 프로그램을 실행해 주는 기능입니다.

예를 들어 코드를 GitHub에 올리거나 Pull Request를 만들면, GitHub Actions가 실행되어 테스트·검사·배포 등의 작업을 수행할 수 있습니다.

1. 코드 수정 및 GitHub에 올리기

2. GitHub Actions 실행

코드 가져오기 → 검사·테스트 → 배포 준비

3. Google Cloud Run에 배포

4. 새로운 서비스 버전 실행

서비스 URL과 /version 응답으로 확인

이 흐름을 정의하는 파일이 워크플로 파일입니다. 보통 저장소의 `.github/workflows/` 폴더에 YAML 형식으로 작성합니다.

③ 워크플로 파일에서 확인할 세 가지

구성 요소	의미	쉬운 비유
트리거(Trigger)	언제 실행할 것인가	자동화 시작 버튼
러너(Runner)	어디에서 실행할 것인가	작업을 수행하는 컴퓨터
단계(Steps)	어떤 작업을 할 것인가	작업 순서표

예를 들어 `main` 브랜치에 코드가 머지되었을 때 실행하도록 설정할 수 있습니다.

- 트리거: `main` 브랜치에 변경 사항이 들어옴
- 러너: GitHub가 제공하는 실행 환경
- 단계: 저장소 코드 가져오기 → 필요한 도구 설치 → 테스트 → Google Cloud 인증 → Cloud Run 배포

중요한 점은 GitHub Actions의 러너는 내 컴퓨터가 아니라 GitHub가 준비한 실행 환경이라는 것입니다.

3. 비유와 실제 사례

Vercel과 GitHub Actions 비교

강의 시작 질문에 답하려면 Vercel이 하는 일을 먼저 이해하면 됩니다.

항목	Vercel	GitHub Actions
실행 계기	저장소의 변경 사항 등	설정한 트리거
코드 확보	연결된 저장소에서 가져옴	워크플로에서 저장소를 가져옴
작업 수행	빌드·검사·배포	설정한 검사·빌드·배포 작업
배포 대상	Vercel 서비스	설정에 따라 Cloud Run 등
권한	GitHub 연결 권한 등	Google Cloud 인증 및 IAM 권한 등

Vercel은 저장소 연결 과정에서 부여받은 권한을 통해 코드를 가져올 수 있습니다. GitHub Actions도 저장소에 접근하는 실행 권한을 이용해 코드를 가져옵니다.

다만 코드를 읽을 권한과 Google Cloud에 서비스를 배포할 권한은 서로 다릅니다. GitHub 저장소를 읽을 수 있다고 해서 Google Cloud에 자동으로 배포할 수 있는 것은 아닙니다.

4. Google Cloud 배포 권한은 어떻게 얻을까요?

이번 강의에서 중요한 보안 개념입니다.

GitHub Actions가 Cloud Run에 배포하려면 Google Cloud가 해당 작업을 수행할 주체를 신뢰하고, 필요한 권한을 부여해야 합니다.

GitHub Actions

자동화 작업을 실행하는 곳

신뢰할 수 있는 인증 방식으로 신원을 확인

Google Cloud 인증 및 IAM

누구에게 어떤 작업을 허용할지 결정

허용된 작업만 수행

Cloud Run

새로운 서비스 버전 배포

키 파일 없이 인증하기

전통적으로는 서비스 계정 키 파일(JSON)을 만들어 GitHub의 비밀 저장소에 보관하고 인증하는 방법을 사용할 수 있습니다. 그러나 키 파일이 유출되면 공격자가 해당 계정의 권한을 악용할 위험이 있습니다.

더 안전한 방법 중 하나는 Workload Identity Federation(WIF) 입니다.

- GitHub Actions가 GitHub에서 발급한 신원 정보를 제시합니다.
- Google Cloud가 해당 신원을 확인합니다.
- 신뢰 관계와 조건이 맞으면 임시 자격 증명을 발급합니다.
- 서비스 계정에 부여된 권한 범위 안에서 배포합니다.

핵심은 키 파일을 장기간 보관하지 않고도 필요한 인증을 수행할 수 있다는 것입니다.

권한 설정에서 반드시 기억할 점

- 프로젝트 전체에 광범위한 권한을 주는 `Owner`나 `Editor` 역할을 무조건 부여하지 않습니다.
- 배포에 필요한 최소 권한만 부여합니다.
- GitHub Actions에서 사용하는 액션은 가능한 한 검증된 버전을 명시합니다.
- 권한 설정이나 리소스 삭제 명령을 실행하기 전에 대상 프로젝트와 영향을 확인합니다.

쉽게 말해, 자동화 직원에게 회사의 모든 열쇠를 주는 대신 맡은 업무에 필요한 열쇠만 주는 것입니다.

5. 브랜치·커밋·PR·머지의 관계

자동 배포를 이해하려면 코드 변경이 어떤 경로로 서비스에 반영되는지도 알아야 합니다.

1

브랜치

기존 서비스에 영향을 주지 않고 작업할 공간

2

커밋

변경 사항을 하나의 기록으로 저장

3

Pull Request (PR)

변경 사항을 검토하고 합치도록 요청

4

머지

검토한 변경 사항을 대상 브랜치에 통합

5

자동 배포

설정된 조건에 따라 새 버전을 배포

여기서 주의할 점은 PR을 만들거나 머지했다고 해서 무조건 배포되는 것은 아니라는 사실입니다. 워크플로 파일에 어떤 트리거와 조건을 설정했는지에 따라 달라집니다.

또한 머지 후 자동 배포가 이루어지더라도, 코드가 정상적으로 작동한다는 사실까지 자동으로 보장되는 것은 아닙니다. 검사와 테스트가 충분하지 않으면 문제가 있는 코드가 사용자에게 전달될 수 있습니다.

6. AI 개발자가 이 내용을 알아야 하는 이유

AI를 활용해 코드를 만드는 능력과 실제 서비스를 안정적으로 운영하는 능력은 다릅니다.

특히 AI 코딩 도구로 코드를 빠르게 수정할수록 다음 문제가 중요해집니다.

1. 재현성: 어떤 코드가 언제 배포되었는지 기록이 남아야 합니다.
2. 안정성: 변경 사항이 배포되기 전에 검사와 테스트를 거쳐야 합니다.
3. 보안: 자동화 시스템에 필요한 최소한의 권한만 부여해야 합니다.
4. 추적성: 실패한 워크플로 단계와 실제 배포된 버전을 연결해 확인할 수 있어야 합니다.
5. 복구 가능성: 잘못된 배포가 발생했을 때 이전의 정상 버전으로 돌아갈 방법이 있어야 합니다.

사용자님의 AI 서비스 구축 목표에서도 중요한 부분입니다. AI가 코드를 작성하고 사람이 결과를 확인하는 것에서 나아가, 검사·배포·버전 확인·복구까지 연결해야 실제로 운영 가능한 서비스가 됩니다.

7. 오늘의 실습에서 확인할 항목

실습을 진행할 때 아래 항목을 하나씩 확인하시면 됩니다.

실습 체크리스트

0/7 완료

- [ ] 워크플로 파일 읽기
  트리거, 러너, 단계가 어디에 정의되어 있는지 찾기
- [ ] 간단한 워크플로 실행
  PR 또는 수동 실행 버튼으로 워크플로 실행하기

- [ ] 실행 로그 확인
  성공·실패한 단계와 오류 메시지 확인하기
- [ ] Google Cloud 인증 구조 확인
  저장소, 서비스 계정, IAM 역할, WIF의 관계 이해하기
- [ ] 권한 설정 전 점검
  키 파일, 과도한 역할, 액션 버전, 삭제 대상 확인하기
- [ ] 실제 배포 확인
  워크플로 로그, Cloud Run 리비전, /version 응답 비교하기
- [ ] 변경 사항 복구 방법 이해
  잘못된 머지나 배포를 되돌리는 방법 확인하기

8. 반드시 기억할 핵심

- CI는 코드 변경 사항을 통합하고 검사하는 자동화 과정입니다.
- CD는 배포할 준비를 하거나 실제 배포까지 자동화하는 과정입니다.
- GitHub Actions는 트리거, 러너, 단계로 구성된 워크플로를 실행합니다.
- GitHub 저장소 접근 권한과 Google Cloud 배포 권한은 서로 다릅니다.
- WIF를 활용하면 장기 보관하는 서비스 계정 키 파일 없이 인증할 수 있습니다.
- 자동 배포는 배포 작업을 자동화할 뿐, 코드의 품질과 안전성을 저절로 보장하지는 않습니다.
- 워크플로 로그, Cloud Run 리비전, `/version` 응답을 연결하면 무엇이 실제로 배포되었는지 확인할 수 있습니다.

9. 시작 질문의 모범 답안

질문: Vercel에 연결된 저장소에 푸시하면 몇 분 뒤 새 버전이 배포되었습니다. 그 몇 분 동안 Vercel은 무엇을 하고 있었을까요? Vercel이 저장소의 코드를 가져갈 수 있었던 것은 어떤 권한 덕분이었을까요?

답변 예시

Vercel은 저장소에 새로운 변경 사항이 올라왔다는 이벤트를 감지한 뒤 코드를 가져와 빌드하고, 필요한 검사와 배포 작업을 수행했을 것입니다. 저장소의 코드를 가져올 수 있었던 것은 GitHub 계정을 연결하는 과정에서 허용한 저장소 접근 권한과 연동 설정 덕분입니다. 이와 마찬가지로 GitHub Actions도 코드를 가져와 자동화 작업을 실행할 수 있지만, Google Cloud에 배포하려면 별도의 인증과 IAM 권한 설정이 필요합니다.

10. 한 문장 요약

GitHub Actions는 코드 변경을 검사하고 배포하는 과정을 자동화하며, 안전한 인증·최소 권한·실행 로그·버전 확인을 함께 설계해야 신뢰할 수 있는 AI 서비스를 운영할 수 있습니다.

Chapter02

/강의정리 챕터 02. 시작 상태 맞추기

1. 학습 목적

이번 강의의 핵심은 자동 배포를 만들기 전에 현재 서비스의 상태를 확인하고, GitHub Actions를 이용해 코드 변경부터 실제 배포까지 이어지는 구조를 이해하는 것입니다.

앞선 챕터 01에서 자동 배포의 필요성을 배웠다면, 이번 챕터에서는 실제로 자동 배포를 시작할 수 있도록 환경을 확인하고, CI/CD와 GitHub Actions의 작동 원리를 배웁니다.

실습 대상은 FastAPI로 만든 AI 친구 에이전트 서비스입니다.

- 서비스: Google Cloud Run에 배포된 `friend-agent`
- API 키: Google Cloud Secret Manager의 `llm-api-key`
- 코드 저장소: GitHub 공개 저장소
- 자동화 도구: GitHub Actions

2. 중학생도 이해하는 핵심 개념

① 왜 시작 상태를 먼저 확인해야 할까요?

자동차를 수리하기 전에 시동이 걸리는지, 연료가 있는지, 문제가 어디에 있는지 확인하는 것과 같습니다.

이미 준비된 환경을 다시 만들면 시간과 비용이 낭비되고, 기존 서비스가 잘못 변경될 수도 있습니다.

확인 항목	확인할 내용	준비되지 않았다면
Google Cloud 설정	로그인, 프로젝트, 기본 리전	시작 상태 문서 A 확인
GitHub 저장소	내 저장소 주소가 연결되어 있는지	문서 A 확인
Cloud Run 서비스	`friend-agent`가 배포되어 있는지	문서 B 확인
API 키 시크릿	`llm-api-key`가 존재하는지	문서 B 확인
서비스 화면	URL 접속 시 캐릭터 만들기 또는 대화 화면이 보이는지	서비스 상태 점검

여기서 시크릿(secret) 은 API 키처럼 외부에 노출되면 안 되는 민감한 값입니다. API 키를 소스 코드에 직접 넣지 않고 Secret Manager에서 관리하면 코드가 공개되어도 키가 함께 노출되는 위험을 줄일 수 있습니다.

② CI와 CD의 차이

CI — 지속적 통합

코드를 합치기 전에 자동으로 테스트하여 문제를 일찍 발견하는 방식입니다.

예: PR을 열면 자동으로 테스트가 실행되고 결과가 표시됩니다.

CD — 지속적 전달 또는 지속적 배포

코드를 배포할 준비를 하거나, 실제 서비스에 반영하는 과정을 자동화합니다.

예: main 브랜치에 머지되면 Cloud Run에 새 버전을 배포합니다.

CD에는 두 가지 방식이 있습니다.

구분	지속적 전달 (Continuous Delivery)	지속적 배포 (Continuous Deployment)
배포 준비	자동화	자동화
실제 배포	사람이 승인할 수 있음	조건을 통과하면 자동 실행
이번 강의	비교 대상	실습 목표

이번 강의에서는 `main` 브랜치에 변경 사항이 들어오면 자동으로 배포되는 구조를 먼저 만듭니다. 이후 사람의 승인 절차나 CI 테스트를 추가하여 안정성을 높이는 방식으로 발전시킬 수 있습니다.

③ GitHub Actions는 무엇인가요?

GitHub Actions는 GitHub 저장소에서 특정 이벤트가 발생했을 때, 미리 작성해 둔 명령을 실행하는 자동화 기능입니다.

예를 들어 다음과 같은 상황에 사용할 수 있습니다.

- `main` 브랜치에 코드가 들어왔을 때
- Pull Request(PR)가 열렸을 때
- 정해진 시각이 되었을 때
- 사용자가 실행 버튼을 눌렀을 때

자동화 절차는 저장소의 `.github/workflows/` 폴더에 YAML 파일로 기록합니다. 이 파일을 워크플로(workflow) 라고 부릅니다.

3. 비유와 실제 사례

수동 배포와 자동 배포 비교

기존: 수동 배포

코드 수정 → 사람이 배포 명령 실행 → Cloud Run 반영

기록과 절차가 개인의 작업 환경에 의존하기 쉬움

개선: GitHub Actions 자동 배포

코드 수정 → GitHub에 커밋·머지 → 자동 워크플로 실행 → Cloud Run 반영

실행 로그와 배포 대상 커밋을 추적할 수 있음

자동 배포의 장점은 단순히 편리하다는 데 있지 않습니다.

- 누가 배포하더라도 정해진 절차를 따릅니다.
- 내 컴퓨터에만 있던 미완성 파일이 배포에 섞일 위험을 줄입니다.
- 언제 어떤 커밋이 배포되었는지 기록을 확인할 수 있습니다.
- 담당자가 자리에 없어도 조건에 맞으면 배포가 진행됩니다.

다만 자동 배포 자체가 코드의 품질을 보장하는 것은 아닙니다. 테스트가 없는 상태에서 잘못된 코드가 머지되면 그 코드도 자동으로 배포될 수 있습니다.

4. 워크플로를 구성하는 핵심 용어

워크플로는 자동화 업무를 적어 둔 작업 지시서라고 생각하시면 됩니다.

용어	쉬운 설명	워크플로에서 확인할 곳
워크플로	자동화 업무 전체	`deploy.yml`
트리거	언제 시작할지	`on:`
작업(Job)	실행할 단계들의 묶음	`jobs:`

용어	쉬운 설명	워크플로에서 확인할 곳
단계(Step)	실제 수행할 개별 작업	`steps:`
러너(Runner)	작업을 실행하는 컴퓨터	`runs-on:`
액션(Action)	다른 사람이 만든 재사용 가능한 작업	`uses:`
변수(Variables)	일반 설정값	`vars`
시크릿(Secrets)	민감한 설정값	`secrets`
워크플로 실행	자동화가 한 번 실행된 기록	GitHub의 Actions 탭

예를 들어 `runs-on: ubuntu-latest`는 GitHub가 제공하는 리눅스 실행 환경을 사용한다는 의미입니다.

이 환경은 보통 워크플로 실행을 위해 새로 준비되므로, 개인 컴퓨터의 미완성 파일이나 로컬 환경 설정에 의존하지 않는 배포를 구성할 수 있습니다.

또한 다음 액션은 Google Cloud Run 배포에 사용하는 액션입니다.

`google-github-actions/deploy-cloudrun@v3`

여기서 `@v3`는 액션 버전을 의미합니다. 버전마다 동작이나 입력값이 달라질 수 있으므로, 실습에서는 템플릿에 지정된 버전을 임의로 변경하지 않는 것이 좋습니다.

5. Google Cloud 인증과 권한

GitHub Actions가 Google Cloud에 배포하려면 인증을 거쳐야 합니다.

GitHub에서 코드를 읽을 수 있는 권한이 있다고 해서 Google Cloud에 배포할 권한까지 자동으로 생기는 것은 아닙니다.

이번 강의에서는 OIDC와 Workload Identity Federation(WIF) 을 활용하는 인증 구조를 배웁니다.

GitHub Actions

자동화 작업 실행

OIDC 신원 토큰

어떤 저장소와 워크플로에서 실행됐는지 증명

Google Cloud WIF 및 IAM

신원을 확인하고 허용된 권한을 부여

Cloud Run 배포

워크플로의 `permissions`에 있는 `id-token: write`는 GitHub Actions가 OIDC 토큰을 요청할 수 있도록 하는 설정입니다. 이것만으로 Google Cloud 배포 권한이 완성되는 것은 아닙니다. Google Cloud 측에서도 신뢰 관계와 IAM 권한을 설정해야 합니다.

실무에서는 다음 원칙을 기억해야 합니다.

- API 키나 서비스 계정 키를 코드에 직접 넣지 않습니다.
- 프로젝트 전체에 광범위한 권한을 부여하는 `Owner`, `Editor` 역할을 무조건 사용하지 않습니다.
- 필요한 권한만 최소한으로 부여합니다.
- 권한 변경이나 삭제 명령을 실행하기 전에 대상과 영향을 확인합니다.

6. AI 서비스에서 자동 배포가 더 중요한 이유

AI 서비스는 일반 웹 서비스보다 작은 변경이 결과에 큰 영향을 줄 수 있습니다.

예를 들어 AI 친구 에이전트에서 다음 항목을 수정한다고 생각해 보겠습니다.

- 시스템 지시문 한 줄
- 사용하는 LLM 모델 이름
- AI가 사용할 도구 설명
- 응답을 선택하는 확신도 기준
- 오류가 발생했을 때의 처리 방식

코드 변경이 작아 보여도 AI의 답변이나 사용자 경험은 크게 달라질 수 있습니다.

따라서 다음 질문에 답할 수 있어야 합니다.

1. 언제 변경되었나요?
2. 어떤 커밋이 배포되었나요?
3. 변경 후부터 AI의 답변이 왜 달라졌나요?
4. 문제가 발생하기 전 버전으로 어떻게 돌아갈 수 있나요?

GitHub Actions의 실행 로그와 Git 커밋 기록, Cloud Run 리비전을 연결해 확인하면 이런 문제를 추적하기가 쉬워집니다.

7. 오늘의 실습: 시작 상태 확인

아래 명령은 기존 환경이 준비되었는지 확인하기 위한 것입니다. 실습 시작 단계에서는 새로운 리소스를 만들거나 설정을 변경하지 않고 상태부터 확인합니다.

```
gcloud config list
git remote -v
gcloud run services describe friend-agent --region us-central1 --format "value(status.
```

```
url)"
gcloud secrets describe llm-api-key --format "value(name)"
```

각 명령의 목적은 다음과 같습니다.

명령	확인하는 내용
`gcloud config list`	로그인 계정, 프로젝트, 기본 설정
`git remote -v`	현재 폴더에 연결된 GitHub 저장소
`gcloud run services describe ...`	Cloud Run 서비스 URL
`gcloud secrets describe ...`	시크릿의 존재 여부

서비스 URL이 확인되면 브라우저에서 열어 캐릭터 만들기나 대화 화면이 표시되는지 확인합니다.

이 명령들은 준비 상태를 확인하는 용도입니다. 시크릿의 실제 API 키 값을 출력하지 않으며, 서비스 URL이 나온다고 해서 애플리케이션의 모든 기능이 정상이라고 단정할 수는 없습니다. 실제 화면도 함께 확인해야 합니다.

8. 객관식 퀴즈 정답과 해설

퀴즈 1. 실제 배포 전에 담당자가 승인하는 방식은?

정답: 2. 지속적 전달(Continuous Delivery)

코드가 배포할 준비를 마치더라도 실제 사용자에게 내보내는 시점은 담당자가 결정할 수 있습니다. 반면 지속적 배포는 정해진 조건을 충족하면 실제 배포까지 자동으로 진행합니다.

퀴즈 2. 작업 브랜치에서 PR만 열었을 때 배포될까요?

정답: 2. 일어나지 않는다. 이 워크플로는 `main` 브랜치에 커밋이 들어올 때만 실행된다.

워크플로의 트리거가 `main` 브랜치의 `push`로 지정되어 있기 때문입니다.

PR이 열렸다는 사실만으로 이 배포 워크플로가 실행되지는 않습니다. 다만 별도로 PR 이벤트를 트리거로 지정한 테스트 워크플로가 있다면 그 워크플로는 실행될 수 있습니다.

9. 반드시 기억할 핵심

- 시작 상태 확인은 불필요한 재설정과 기존 서비스 훼손을 방지합니다.
- CI는 변경 사항을 통합하기 전에 자동으로 테스트하는 방식입니다.
- CD는 배포 준비 또는 실제 배포를 자동화하는 방식입니다.
- GitHub Actions는 트리거, 러너, 작업, 단계로 구성됩니다.

- OIDC와 WIF를 사용하면 장기 보관하는 서비스 계정 키 없이 Google Cloud 인증을 구성할 수 있습니다.
- 자동 배포는 편리하지만, 충분한 테스트가 없다면 잘못된 코드도 사용자에게 전달할 수 있습니다.
- AI 서비스에서는 작은 변경도 응답에 영향을 주므로 커밋과 배포 기록을 추적하는 것이 중요합니다.

10. 한 문장 요약

GitHub Actions로 자동 배포를 만들기 전에 기존 서비스와 인증 환경을 확인하고, CI/CD·워크플로·최소 권한 인증의 원리를 이해해야 안전하고 추적 가능한 AI 서비스 운영이 가능합니다.

Chapter03

/강의정리 챕터 03. main에 바로 고치지 않는 이유

1. 학습 목적

이번 강의의 핵심은 안전하게 코드를 변경하는 방법과 GitHub Actions가 실행되는 과정을 직접 경험하는 것입니다.

앞선 챕터에서 자동 배포의 구조를 배웠다면, 이번에는 작업 브랜치와 PR을 이용해 변경 사항을 관리하고, 실제 워크플로를 만들어 실행·실패·복구까지 연습합니다.

오늘 배우는 내용은 크게 세 가지입니다.

1. Git 협업 흐름: 브랜치 → 커밋 → 푸시 → PR → 머지
2. GitHub Actions 실행: 워크플로 작성, 트리거, 러너, 단계별 로그 확인
3. 실패와 복구: 실패한 단계 확인, `git revert`로 변경 취소

2. 중학생도 이해하는 핵심 개념

① 왜 main에서 직접 고치지 않을까요?

`main`은 사용자에게 제공할 서비스의 기준이 되는 브랜치입니다. 이번 실습에서는 `main`에 커밋이 들어오면 자동 배포되는 구조를 만들 예정이므로, 잘못된 코드가 곧바로 사용자에게

전달될 수 있습니다.

학교 신문을 만든다고 생각해 보세요. 인쇄 중인 최종 신문을 직접 수정하는 대신, 초안을 따로 만들어 내용을 검토한 뒤 최종본에 반영하는 것이 안전합니다.

작업 브랜치

새로운 기능이나 수정 사항을 별도로 작업하는 공간입니다.

Pull Request (PR)

변경한 내용을 확인하고 main에 합쳐 달라고 요청합니다.

머지(Merge)

검토가 끝난 변경 사항을 main에 반영합니다.

이번 실습의 자동 배포 워크플로에서는 main에 변경 사항이 들어오면 배포가 시작됩니다.

② Git의 주요 용어

용어	쉬운 설명
브랜치(Branch)	독립적으로 작업하는 공간
커밋(Commit)	변경 사항을 기록하는 것
푸시(Push)	내 컴퓨터의 커밋을 GitHub에 올리는 것
PR(Pull Request)	변경 사항을 검토하고 합쳐 달라고 요청하는 것
머지(Merge)	변경 사항을 대상 브랜치에 통합하는 것
`git revert`	기존 커밋의 변경을 취소하는 새 커밋을 만드는 것

특히 푸시와 머지는 서로 다른 작업입니다.

작업 브랜치에 푸시했다고 해서 그 코드가 `main`에 합쳐진 것은 아닙니다. GitHub에 변경 사항이 올라갔을 뿐이며, PR을 통해 검토하고 머지해야 `main`에 반영됩니다.

③ PR을 사용하는 이유

PR 화면에서는 다음 정보를 확인할 수 있습니다.

- 어떤 파일을 변경했는지
- 어떤 줄을 추가하거나 삭제했는지
- 변경 목적과 설명이 무엇인지
- 자동 테스트가 성공했는지 실패했는지
- 언제 어떤 변경 사항이 머지되었는지

이 기록은 나중에 AI의 답변이 갑자기 달라졌거나 서비스 오류가 발생했을 때 원인을 찾는 데 도움이 됩니다.

3. 첫 번째 실습: 브랜치에서 README 수정하기

이번 실습은 서비스 기능에 영향을 주지 않는 작은 변경부터 시작합니다.

README.md 맨 아래에 저장소의 목적을 설명하는 한 줄을 추가합니다.

```
# 최신 main으로 시작
git switch main
git pull

# 작업 브랜치 생성
git switch -c readme-owner-note

# 변경한 파일을 기록
git add README.md
git commit -m "README에 저장소 설명 한 줄 추가"

# GitHub에 작업 브랜치 업로드
git push -u origin readme-owner-note

# PR 생성
gh pr create --base main \
  --title "README에 저장소 설명 한 줄 추가" \
  --body "README.md 맨 아래에 저장소 설명을 한 줄 추가한다"
```

이 명령들은 각각 브랜치 생성, 변경 기록, GitHub 업로드, PR 생성을 수행합니다.

여기서 중요한 실무 원칙은 PR을 만들었다고 바로 머지하지 않는 것입니다. 먼저 브라우저에서 `Files changed` 탭을 열어 실제 변경 사항이 의도한 내용인지 확인해야 합니다.

푸시와 PR 생성은 GitHub 저장소를 실제로 변경하는 작업입니다. 코딩 에이전트에게 맡길 때도 실행 전에 대상 브랜치와 명령을 확인하고, `main`으로 직접 푸시하거나 의도하지 않은 강제 푸시를 하지 않도록 주의합니다.

4. 두 번째 실습: 첫 워크플로 만들기

이제 GitHub Actions가 실제로 실행되는 과정을 경험합니다.

Google Cloud에 배포하는 작업은 아직 만들지 않습니다. 대신 `hello.yml`이라는 연습용 워크플로를 만들어 러너의 정보를 확인합니다.

파일 위치는 다음과 같습니다.

```
.github/workflows/hello.yml
```

워크플로의 핵심 설정은 다음과 같습니다.

설정	의미
`name: hello`	워크플로 이름
`pull_request:`	PR 이벤트가 발생하면 실행
`workflow_dispatch:`	사용자가 직접 실행할 수 있도록 허용
`permissions: contents: read`	저장소 내용을 읽는 권한만 허용
`runs-on: ubuntu-latest`	Ubuntu 실행 환경 사용
`actions/checkout@v7`	GitHub 저장소의 코드를 러너로 가져오기

이 워크플로는 두 가지 상황에서 실행됩니다.

- PR을 열거나 PR에 새 커밋을 추가했을 때
- Actions 탭에서 `Run workflow` 버튼을 눌렀을 때

반면, 이 워크플로에는 Google Cloud 인증이나 배포 단계가 없으므로 Cloud Run 서비스를 변경하지 않습니다.

5. 러너(Runner)란 무엇인가요?

러너는 GitHub Actions의 명령을 실제로 실행하는 컴퓨터입니다.

내 컴퓨터에서 코드를 실행하는 것과 달리, GitHub Actions는 별도의 실행 환경에서 저장소의 코드를 가져와 작업합니다.

내 컴퓨터

커밋하지 않은 파일이나 `.env` 같은 로컬 설정 파일이 있을 수 있습니다.

GitHub Actions 러너

저장소에 커밋되어 GitHub에 올라간 코드를 가져와 작업합니다.

실행 로그에서 코드를 내려받기 전과 후의 파일 목록을 비교하면 이 차이를 확인할 수 있습니다.

- 코드를 내려받기 전에는 작업 공간이 거의 비어 있습니다.
- `actions/checkout@v7` 단계가 실행되면 저장소 파일이 나타납니다.
- `.env`가 `.gitignore`에 등록되어 커밋되지 않았다면 러너에 자동으로 복사되지 않습니다.

이것이 중요한 이유는 배포 대상이 내 컴퓨터의 현재 폴더가 아니라 GitHub에 올라간 코드로 통제되기 때문입니다.

단, `.env`가 이미 커밋되었거나 다른 방법으로 러너에 전달되도록 설정되어 있다면 별도 확인이 필요합니다. `.gitignore`는 파일의 업로드를 자동으로 차단하는 보안 장치가 아니라, Git이 추적하지 않도록 돕는 설정입니다.

6. 세 번째 실습: 실패한 워크플로 확인하기

이번에는 일부러 워크플로를 실패시켜 봅니다.

`러너 정보 보기` 단계의 `run` 명령 맨 아래에 다음 한 줄을 추가합니다.

```
exit 1
```

`exit 1`은 해당 명령이 실패 상태로 종료되었다고 알리는 명령입니다.

커밋하고 푸시하면 PR에 새로운 변경이 추가되므로 워크플로가 다시 실행됩니다.

```
git commit -am "hello 워크플로 실패 확인"
git push
gh run watch
```

실행 로그에서 다음 결과를 확인합니다.

- `러너 정보 보기` 단계가 실패합니다.
- 로그에 `Process completed with exit code 1`이 표시됩니다.
- 뒤에 있는 코드 다운로드 및 파일 확인 단계는 일반적으로 건너뜁니다.
- PR의 상태 체크가 빨간 X로 바뀝니다.

여기서 매우 중요한 점이 있습니다.

상태 체크가 실패했다고 해서 머지가 자동으로 차단되는 것은 아닙니다. 저장소의 브랜치 보호 규칙이나 규칙 세트에서 해당 상태 체크를 필수로 설정해야 머지를 막을 수 있습니다.

실패를 확인한 다음에는 `git revert`로 실패를 유발한 커밋을 되돌립니다.

```
git revert --no-edit HEAD
git push
gh run watch
```

`git revert`는 과거 기록을 지우는 대신 변경을 취소하는 새 커밋을 만듭니다. 따라서 어떤 변경이 있었고 어떻게 복구했는지도 기록으로 남습니다.

7. 네 번째 실습: 직접 실행하기

실패를 되돌려 초록 체크가 나타나는 것을 확인했다면 PR을 머지합니다.

```
gh pr merge --merge --delete-branch
git switch main
git pull

gh workflow run hello.yml
gh run watch
```

이 명령은 PR을 머지하고, 로컬의 `main`을 최신 상태로 맞춘 다음, `hello.yml`을 수동으로 실행합니다.

`workflow_dispatch`가 설정되어 있으므로 Actions 탭에서 `Run workflow` 버튼을 누르는 방법도 가능합니다.

로그에서 이번 실행의 이벤트 이름이 `workflow_dispatch`로 나타나는지 확인합니다.

이렇게 하면 같은 워크플로라도 어떤 이벤트가 실행을 시작했는지에 따라 로그에 다른 이벤트 이름이 기록된다는 것을 이해할 수 있습니다.

8. 객관식 퀴즈 정답과 해설

퀴즈 1. 작업 브랜치에서 `git push`까지 끝냈지만 PR은 아직 열지 않았습니다. 변경 사항은 어디에 있을까요?

정답: 2. GitHub의 작업 브랜치에 올라가 있지만 main에는 아직 합쳐지지 않았다.

푸시는 로컬 커밋을 원격 저장소에 업로드하는 작업입니다. `main`에 통합하려면 별도의 머지 과정이 필요합니다.

퀴즈 2. `.env 없음`이 출력되는 이유는 무엇인가요?

정답: 2. 러너는 GitHub에 올라간 커밋만 내려받고, `.env`는 `.gitignore`에 적혀 있어 커밋되지 않았기 때문이다.

GitHub Actions 러너는 내 컴퓨터의 파일을 그대로 사용하는 것이 아니라 저장소의 커밋을 내려받습니다. 따라서 로컬에만 있는 `.env`는 자동으로 전달되지 않습니다.

9. AI 개발자가 반드시 기억할 점

이번 강의는 단순히 Git 명령을 익히는 과정이 아닙니다. AI 서비스를 안전하게 개발하고 운영하기 위한 기본적인 업무 방식입니다.

- 브랜치: 변경 사항을 분리해 작업합니다.
- PR: 변경 내용을 확인하고 검토 기록을 남깁니다.
- 워크플로: 반복 작업을 정해진 절차로 자동 실행합니다.
- 러너: 개인 컴퓨터와 분리된 환경에서 코드를 실행합니다.
- 로그: 단계별 성공과 실패를 확인합니다.
- revert: 잘못된 변경을 기록을 남기면서 되돌립니다.
- 필수 상태 체크: 테스트가 실패한 코드의 머지를 차단하도록 설정할 수 있습니다.

사용자님이 구축하시는 AI 에이전트나 My-Story-Maker 같은 서비스에도 동일한 원리를 적용할 수 있습니다. AI의 지시문이나 모델 설정을 수정하더라도 작업 브랜치에서 검토하고, 테스트를 통과한 변경만 서비스에 반영하는 흐름을 만드는 것입니다.

10. 한 문장 요약

`main`을 직접 수정하지 않고 브랜치와 PR로 변경을 관리하며, GitHub Actions의 실행 로그와 실패·복구 과정을 익히는 것이 안전한 AI 서비스 자동화의 출발점입니다.

Chapter04

/강의정리 챕터 04. GitHub Actions의 Google Cloud 배포 권한 설정

1. 학습 목적

이번 강의의 핵심은 GitHub Actions가 서비스 계정 키 파일 없이도 Google Cloud에 안전하게 배포하도록 권한을 설정하는 것입니다.

지난 챕터에서는 GitHub Actions의 실행 환경인 러너, 브랜치와 PR, 워크플로 로그와 실패 복구를 배웠습니다. 이번에는 한 걸음 더 나아가, GitHub Actions가 실제 Google Cloud에 배포할 수 있도록 신원을 확인하고 필요한 권한만 부여하는 구조를 배웁니다.

붙여넣은 텍스트(1).txt

이번 챕터에서 꼭 이해해야 할 개념은 다음 네 가지입니다.

- 서비스 계정(Service Account): 프로그램이 사용하는 Google Cloud 계정

- WIF(Workload Identity Federation): 장기 키 파일 없이 외부 프로그램의 신원을 확인하는 방식
- 최소 권한 원칙: 각 계정에 맡은 일에 필요한 권한만 부여하는 원칙
- 서비스 계정 가장(Impersonation): 다른 서비스 계정에 부여된 권한을 일정 시간 빌려 사용하는 방식

2. 중학생도 이해할 수 있는 설명

2-1. 왜 GitHub Actions에 별도의 배포 권한이 필요할까요?

GitHub Actions가 실행되는 러너는 Google Cloud 안에 있는 컴퓨터가 아닙니다. 따라서 Google Cloud 입장에서는 러너가 누구인지, 정말 내 저장소에서 실행된 작업인지 확인해야 합니다.

붙여넣은 텍스트(1).txt

학교 건물에 비유해 보겠습니다.

- GitHub Actions 러너는 학교에 방문한 외부 작업자입니다.
- Google Cloud는 학교 건물입니다.
- 서비스 계정은 특정 업무를 할 수 있는 출입증입니다.
- WIF는 방문자의 신분증을 확인하는 출입 관리 시스템입니다.
- IAM 권한은 출입증으로 들어가서 할 수 있는 업무의 범위입니다.

외부 작업자에게 건물의 모든 열쇠를 주는 대신, 신원을 확인하고 맡은 업무에 필요한 출입 권한만 주는 것입니다.

2-2. WIF는 무엇인가요?

WIF는 Workload Identity Federation의 약자입니다. 쉽게 말하면, GitHub Actions가 장기간 보관해야 하는 비밀번호나 키 파일 없이 Google Cloud에 자신의 신원을 증명하는 방법입니다.

기존 방식처럼 서비스 계정 키 파일을 만들어 GitHub에 보관하면 키가 유출될 경우 계속 악용될 위험이 있습니다. WIF 방식에서는 배포할 때마다 GitHub가 발급한 OIDC 토큰을 이용해 신원을 확인하고, 조건을 통과하면 제한된 시간 동안 사용할 수 있는 액세스 토큰을 발급받습니다.

붙여넣은 텍스트(1).txt

핵심은 다음과 같습니다.

- 키 파일을 장기간 저장하지 않습니다.

- 지정한 GitHub 저장소에서 온 요청인지 확인합니다.
- 승인된 신원만 배포 서비스 계정의 권한을 사용할 수 있습니다.
- 액세스 토큰은 일정 시간이 지나면 만료됩니다.

3. 배포 과정은 어떻게 작동할까요?

① GitHub Actions 러너

배포할 때마다 GitHub에 OIDC 토큰을 요청합니다.

토큰 제출

② WIF 공급자: 저장소 확인

GitHub가 서명한 토큰인지, 토큰에 기록된 저장소가 지정한 저장소와 일치하는지 확인합니다.

첫 번째 검사 통과

③ 배포 서비스 계정 권한 확인

해당 저장소의 신원이 `github-deployer` 계정을 사용할 권한이 있는지 확인합니다.

두 번째 검사 통과 · 액세스 토큰 발급

④ Google Cloud 배포

발급된 액세스 토큰으로 배포 서비스 계정에 부여된 권한만 사용합니다.

이 과정에서 검사가 두 번 이루어지는 이유가 중요합니다.

검사	확인하는 내용
첫 번째: 공급자 조건	이 요청을 Google Cloud에서 받아들여도 되는가?
두 번째: 권한 연결	이 신원이 `github-deployer` 계정을 사용할 수 있는가?

즉, 신원을 인정받는 것과 특정 계정의 권한을 사용할 수 있는 것은 별개입니다.

붙여넣은 텍스트(1).txt

4. 서비스 계정을 세 개로 나누는 이유

이번 강의에서는 세 가지 서비스 계정을 구성합니다. 각각의 역할을 구분하는 것이 중요합니다.

① `friend-agent-runtime` — 실행 담당

Cloud Run에서 서비스가 실제로 실행될 때 사용합니다. 이 강의에서는 `llm-api-key` 시크릿 하나를 읽을 수 있는 권한을 부여합니다.

② `friend-agent-builder` — 빌드 담당

Cloud Build가 소스 코드를 컨테이너 이미지로 만드는 데 사용합니다. 빌드에 필요한 역할을 부여하고 배포 계정이 이 계정을 사용해 빌드를 실행할 수 있도록 설정합니다.

③ `github-deployer` — 배포 담당

GitHub Actions가 Google Cloud에 배포할 때 사용합니다. 배포에 필요한 역할과 런타임·빌드 서비스 계정을 지정해 사용할 권한을 부여받습니다.

왜 하나의 계정으로 통합하면 안 될까요?

예를 들어 실행 계정과 배포 계정을 하나로 합치면, 외부 요청을 처리하는 서비스가 배포 권한까지 보유하게 될 수 있습니다. 서비스의 코드나 설정에 문제가 생겼을 때 피해 범위가 커질 수 있습니다.

따라서 각 계정의 권한을 분리해야 합니다. 이것이 최소 권한 원칙(Principle of Least Privilege) 입니다.

붙여넣은 텍스트(1).txt

5. 이번 강의에서 만드는 여섯 가지 설정

순서	설정 항목	목적
1	런타임 서비스 계정	서비스 실행과 시크릿 읽기
2	빌드 서비스 계정	소스 빌드에 필요한 권한 분리
3	배포 서비스 계정	배포 업무에 필요한 권한 부여
4	WIF 풀 `github`	외부에서 오는 신원을 받아들일 묶음 생성
5	공급자 `friend-agent-repo` 및 권한 연결	지정한 저장소만 배포 계정을 사용할 수 있게 제한
6	GitHub 저장소 변수 다섯 개	워크플로가 프로젝트·공급자·계정 정보를 읽도록 설정

붙여넣은 텍스트(1).txt

GitHub에 등록하는 변수 다섯 개

강의에서는 다음 값들을 GitHub 저장소의 Settings → Secrets and variables → Actions → Variables에 등록합니다.

- `GCP_PROJECT_ID`: Google Cloud 프로젝트 ID
- `WIF_PROVIDER`: WIF 공급자 이름
- `DEPLOY_SA`: 배포 서비스 계정 주소

- `RUNTIME_SA`: 실행 서비스 계정 주소
- `BUILD_SA`: 빌드 서비스 계정 주소

이 값들은 강의에서 비밀이 아닌 설정 정보로 분류하므로 `Secrets`가 아닌 `Variables`에 저장합니다. 단, 실제 API 키 값 같은 비밀 정보는 여기에 넣지 않습니다.

붙여넣은 텍스트(1).txt

6. 코딩 에이전트에게 권한 설정을 맡길 때의 안전 수칙

이번 강의는 단순히 명령을 실행하는 방법보다 AI 에이전트가 권한을 잘못 설정하지 않도록 통제하는 방법도 강조합니다.

특히 권한 오류가 발생했을 때 에이전트가 더 강한 권한을 부여하는 방식으로 문제를 해결하려 할 수 있으므로, 명령을 실행하기 전에 확인해야 합니다.

서비스 계정 키 파일을 만들지 않습니다.

키 파일 생성 명령이 있으면 실행하지 않고 WIF 방식으로 설정합니다.

Owner, Editor 같은 광범위한 역할을 함부로 부여하지 않습니다.

강의 문서에 명시된 역할과 대조하고, 권한 오류를 해결하기 위해 권한 범위를 임의로 확대하지 않습니다.

권한은 필요한 대상에만 부여합니다.

시크릿 읽기 권한은 `llm-api-key` 하나에, `serviceAccountUser` 권한은 지정된 런타임·빌드 서비스 계정에만 부여합니다.

공급자 조건에 정확한 저장소 이름을 사용합니다.

저장소 소유자 이름만 지정하면 같은 계정의 다른 저장소까지 배포 권한을 얻을 수 있습니다.

붙여넣은 텍스트(1).txt

코딩 에이전트에 지시할 때 활용할 프롬프트

Google Cloud와 GitHub Actions의 배포 권한을 설정해 주세요.

1. 제공된 `docs/wif-setup.md`의 절차와 역할만 사용하세요.
2. 서비스 계정 키 파일은 절대 만들지 마세요.
3. Owner, Editor 등 문서에 없는 광범위한 권한을 추가하지 마세요.
4. 시크릿의 실제 값은 읽거나 출력하지 마세요.

5. 명령을 실행하기 전에 무엇을 만들고 누구에게 어떤 권한을 주는지 설명하고 제 승인을 받으세요.
6. WIF 공급자 조건과 권한 연결에 지정된 GitHub 저장소가 모두 정확한지 실행 전에 보여 주세요.
7. 설정 후 서비스 계정별 역할, 공급자 저장소 조건, GitHub 저장소 변수 다섯 개를 확인해 결과를 보고하세요.

오류가 발생해도 권한을 임의로 확대하지 말고, 오류 메시지와 현재 설정을 먼저 확인해 주세요.

7. 설정 후 반드시 확인할 사항

설정을 마쳤다고 해서 바로 안전하다고 판단하면 안 됩니다. 강의에서는 Google Cloud 콘솔과 GitHub 저장소에서 다음 항목을 확인하도록 안내합니다.

붙여넣은 텍스트(1).txt

- [ ] WIF 공급자 `friend-agent-repo`의 조건에 정확한 저장소가 지정되어 있는가?
- [ ] `github-deployer`, `friend-agent-runtime`, `friend-agent-builder` 서비스 계정에 키 파일이 없는가?
- [ ] 배포 서비스 계정에 강의에서 지정한 배포 관련 역할이 부여되어 있는가?
- [ ] GitHub 저장소 변수 다섯 개가 등록되어 있는가?
- [ ] 서비스 계정별 권한이 필요한 범위에만 부여되어 있는가?

주의할 점도 있습니다. IAM 변경은 즉시 반영되지 않을 수 있으며, 강의에서는 전파에 최대 약 5분이 걸릴 수 있다고 설명합니다. 인증 오류가 발생하면 잠시 기다린 뒤 재실행하고, 계속 실패하면 오류에 나온 저장소와 계정 이름이 설정값과 일치하는지 확인합니다.

붙여넣은 텍스트(1).txt

8. 객관식·주관식 퀴즈 정답

객관식

Q. 공급자 규칙을 특정 저장소 하나가 아니라 GitHub 계정 전체로 설정하면 어떻게 될까요?

정답: 2번. 해당 GitHub 계정의 모든 저장소에서 실행되는 워크플로가 배포 서비스 계정을 가장할 수 있게 됩니다.

이유: 저장소 하나만 허용하는 조건과 계정 전체를 허용하는 조건은 권한 범위가 다르기 때문입니다.

붙여넣은 텍스트(1).txt

주관식

Q. `github-deployer`와 `friend-agent-runtime`을 하나로 합치면 무엇이 달라질까요?

모범 답안:

`github-deployer`는 GitHub Actions가 배포할 때 사용하는 계정이고, `friend-agent-runtime`은 Cloud Run 서비스가 실행되는 동안 사용하는 계정입니다. 두 계정을 하나로 합치면 실행 중인 서비스가 배포 권한까지 갖게 될 수 있고, 서비스에 문제가 생겼을 때 피해 범위가 커집니다. 계정을 분리하면 배포 업무와 서비스 실행 업무의 권한을 각각 제한할 수 있어 보안성이 높아집니다.

붙여넣은 텍스트(1).txt

9. 내 AI 서비스 프로젝트에 적용한다면?

사용자용 AI 에이전트나 MSM(My-Story-Maker)처럼 사용자의 글을 분석하고 결과를 생성하는 서비스를 만든다고 가정해 보겠습니다.

- 실행 계정: AI 모델 API 키를 읽고 사용자 요청을 처리합니다.
- 빌드 계정: 서비스 코드를 컨테이너 이미지로 만듭니다.
- 배포 계정: 새 버전의 서비스를 Google Cloud에 배포합니다.
- WIF: 지정한 GitHub 저장소의 워크플로만 배포 계정을 사용할 수 있도록 신원을 확인합니다.

이 구조를 사용하면 서비스가 사용자 요청을 처리하는 데 필요한 권한과 새 버전을 배포하는 데 필요한 권한을 분리할 수 있습니다.

1인 AI 사업에서도 이 구분은 중요합니다. 혼자 개발하더라도 AI 코딩 에이전트가 실행하는 명령이 클라우드의 권한을 변경할 수 있기 때문입니다. AI에게 일을 맡기는 능력만큼, AI가 어디까지 할 수 있는지를 제한하는 능력도 중요합니다.

10. 반드시 기억할 핵심

- WIF: 장기 서비스 계정 키 파일 없이 외부 프로그램의 신원을 확인하는 방식입니다.
- 두 번의 검사: 공급자는 들어오는 신원을 확인하고, 권한 연결은 그 신원이 사용할 서비스 계정을 확인합니다.
- 서비스 계정 분리: 실행·빌드·배포 권한을 분리해 문제가 발생했을 때 피해 범위를 줄입니다.
- 최소 권한 원칙: 필요한 대상에 필요한 권한만 부여합니다.

- 정확한 저장소 조건: GitHub 계정 전체가 아니라 지정한 저장소만 허용해야 합니다.
- AI 에이전트 통제: 키 파일 생성, 광범위한 권한 부여, 임의의 권한 확대를 막고 명령 실행 전에 검토합니다.

한 문장 요약: GitHub Actions가 Google Cloud에 안전하게 배포하도록 WIF로 신원을 확인하고, 실행·빌드·배포 서비스 계정을 분리해 필요한 권한만 부여하는 것이 이번 챕터의 핵심입니다.

Chapter05

/강의정리 챕터 05. 워크플로 파일 넣기

1. 학습 목적

이번 강의의 목표는 GitHub Actions에 자동 배포 워크플로를 등록하고, 실제로 Google Cloud에 배포되는지 확인하는 것입니다.

앞선 챕터에서는 배포에 필요한 권한을 설정했습니다. 이번에는 그 권한을 사용하는 워크플로 파일을 저장소에 넣고, 실제 배포와 서비스 실행까지 확인합니다. 마지막으로 기존 기본 서비스 계정에 남아 있는 불필요한 API 키 읽기 권한도 회수합니다.

이번 강의의 전체 흐름은 다음과 같습니다.

1. 워크플로 파일 복사

   `starter/workflows/deploy.yml`을 `.github/workflows/deploy.yml`로 복사합니다.

2. 브랜치에서 PR 생성

   변경 내용을 검토할 수 있도록 작업 브랜치에서 PR을 만듭니다.

3. PR 머지 및 자동 배포

   `main`에 변경 사항이 반영되면 GitHub Actions가 배포를 시작합니다.

4. 배포 결과 확인

   GitHub Actions 로그, Cloud Run 리비전, 서비스 계정, `/version`을 확인합니다.

5. 불필요한 권한 회수

   새 런타임 계정이 정상 작동하는 것을 확인한 뒤 기존 기본 계정의 시크릿 읽기 권한을 제거합니다.

2. 왜 워크플로 파일을 특정 폴더에 넣어야 할까요?

GitHub 저장소에 YAML 파일이 있다고 해서 모두 자동으로 실행되는 것은 아닙니다.

- `starter/workflows/deploy.yml`: 워크플로 템플릿을 보관하는 파일입니다.
- `.github/workflows/deploy.yml`: GitHub Actions가 워크플로로 인식하고 실행하는 위치입니다.

따라서 이번 실습에서는 템플릿 파일을 실행 위치로 복사합니다. 중요한 점은 파일 내용을 수정하는 것이 아니라, 검토된 템플릿을 지정된 위치에 등록하는 것입니다.

직접 실행하는 명령

```
git switch main
git pull
git switch -c add-deploy-workflow

mkdir -p .github/workflows
cp starter/workflows/deploy.yml .github/workflows/deploy.yml

git add .github/workflows/deploy.yml
git commit -m "main 머지 시 Cloud Run 자동 배포"
git push -u origin add-deploy-workflow

gh pr create --base main \
  --title "main 머지 시 Cloud Run 자동 배포" \
  --body "starter/workflows/deploy.yml 을 그대로 복사한다"
```

여기서 중요한 것은 마지막에 PR을 만든다는 점입니다. PR 생성과 머지는 다릅니다. PR을 만들었다고 `main`에 반영되거나 자동 배포가 시작되는 것은 아닙니다.

3. PR 머지 전에 확인해야 할 다섯 가지

이번 PR은 일반적인 문서 수정과 다릅니다. 머지하는 순간 `main`에 새 커밋이 생기고, 그 커밋이 자동 배포를 시작하기 때문입니다.

확인 항목	확인할 내용
`on`	`main`에 대한 `push` 이벤트만 지정되어 있는가?
`permissions`	`contents: read`, `id-token: write`만 설정되어 있는가?
`uses`	`actions/checkout@v7`, `google-github-actions/auth@v3`, `google-github-actions/deploy-cloudrun@v3`인가?
`service`, `region`	서비스 이름이 `friend-agent`, 리전이 `us-central1`인가?
서비스 계정	런타임은 `RUNTIME_SA`, 빌드는 `BUILD_SA`를 지정하는가?

특히 액션 버전, 서비스 이름, 리전은 임의로 바꾸지 않습니다. 저장소마다 다른 프로젝트 ID와 서비스 계정 등의 값은 앞서 설정한 저장소 변수를 사용하므로, 템플릿을 수정할 필요가 없습니다.

`COMMIT_SHA`는 왜 필요할까요?

워크플로는 다음과 같은 정보를 서비스 환경변수로 전달합니다.

```
COMMIT_SHA: ${{ github.sha }}
```

`github.sha`는 해당 실행을 일으킨 커밋의 고유 식별값입니다. 서비스의 `/version` 경로에서 이 값을 확인하면 현재 실행 중인 서비스가 어떤 커밋을 기반으로 배포되었는지 확인할 수 있습니다.

예를 들어 GitHub의 최신 커밋 SHA와 `/version`의 `commit` 값이 같다면, 배포된 코드 버전을 확인하는 데 도움이 됩니다.

4. 머지 후 자동 배포 지켜보기

PR의 변경 사항을 검토한 뒤 머지하면 자동 배포가 시작됩니다.

```
gh pr merge --merge --delete-branch
gh run watch
```

첫 번째 명령은 PR을 머지하고 작업 브랜치를 삭제합니다. 두 번째 명령은 시작된 GitHub Actions 실행을 관찰합니다.

GitHub 웹 화면에서도 `Actions` 탭을 열어 실행 상태를 확인할 수 있습니다. 각 단계의 로그를 보면 어느 과정까지 성공했는지, 실패했다면 어느 단계에서 문제가 생겼는지 알 수 있습니다.

배포 과정의 단계별 의미

Checkout

배포할 저장소 코드를 러너로 가져옵니다.

Auth

WIF를 통해 Google Cloud에 인증하고 배포에 사용할 권한을 얻습니다.

Deploy

소스 빌드와 이미지 저장 등의 과정을 거쳐 Cloud Run에 새 리비전을 배포합니다.

배포 주소 확인

배포된 서비스의 주소를 확인합니다.

5. 실패했을 때 어디부터 확인할까요?

실패 메시지를 보고 곧바로 권한을 확대하지 않는 것이 중요합니다.

실패 위치 또는 메시지	우선 확인할 내용
인증 단계에서 `Permission` 또는 `unauthorized`	IAM 변경 전파가 끝났는지, 공급자 조건의 저장소 이름이 정확한지 확인합니다.
빌드 서비스 계정 관련 권한 오류	`github-deployer`가 `friend-agent-builder`를 사용할 권한이 있는지, `BUILD_SA` 변수가 있는지 확인합니다.
배포 실패	실패한 단계의 로그를 읽고 원인을 파악합니다.

인증 오류라면 몇 분 기다렸다가 다시 실행해 볼 수 있습니다. 빌드 서비스 계정 권한이 문제라면 앞선 챕터에서 설정한 권한과 저장소 변수를 확인해야 합니다.

주의: 기본 서비스 계정을 사용할 수 있도록 권한을 추가하는 방식으로 문제를 해결하지 않습니다. 기본 계정에는 광범위한 `Editor` 권한이 붙어 있을 수 있기 때문입니다.

실패했을 때는 먼저 원인을 파악하고, 수정이 필요한 경우에도 무엇을 바꿀지 검토한 뒤 진행합니다.

6. 배포 성공 여부를 네 가지 관점에서 확인하기

GitHub Actions에서 모든 단계가 성공했다고 끝난 것은 아닙니다. 실제 서비스가 올바른 설정으로 실행되는지도 확인해야 합니다.

① Cloud Run 리비전과 트래픽

`Cloud Run → friend-agent → 리비전`에서 새 리비전이 생성되었는지, 해당 리비전이 트래픽을 100% 받는지 확인합니다.

② 런타임 서비스 계정

새 리비전의 보안 설정에서 실행 계정이 기본 계정이 아니라 `friend-agent-runtime`인지 확인합니다.

③ 빌드 서비스 계정

`Cloud Build → 기록`에서 최신 빌드의 실행 계정이 `friend-agent-builder`인지 확인합니다.

④ 실제 배포된 커밋

브라우저에서 `<서비스 URL>/version`을 열어 `commit` 값을 GitHub 저장소의 최신 커밋 SHA와 비교합니다.

마지막으로 서비스에 직접 접속해 캐릭터와 한두 마디 대화해 봅니다. 새 런타임 계정으로 바뀐 뒤에도 LLM API 키를 읽고 정상적으로 답하는지 확인해야 합니다.

7. 기본 서비스 계정의 시크릿 읽기 권한 회수하기

새 리비전이 `friend-agent-runtime` 계정으로 정상 실행되고, 실제 답변까지 반환하는 것을 확인했다면 다음 단계로 넘어갑니다.

초기 설정에서 기본 서비스 계정에 부여했던 `llm-api-key` 읽기 권한을 더 이상 유지할 필요가 없습니다. 권한을 제거하면 기본 계정으로 실행되는 다른 서비스가 해당 시크릿을 읽을 수 있는 경로도 줄어듭니다.

권한 회수 명령

```
gcloud secrets remove-iam-policy-binding llm-api-key \
  --project $PROJECT_ID \
  --member "serviceAccount:$PROJECT_NUMBER-compute@developer.gserviceaccount.com" \
  --role roles/secretmanager.secretAccessor

gcloud secrets get-iam-policy llm-api-key \
  --project $PROJECT_ID
```

첫 번째 명령은 기본 서비스 계정의 읽기 권한을 제거하고, 두 번째 명령은 남아 있는 권한을 확인합니다.

이 명령은 권한을 삭제하는 작업이므로 실행 전에 대상 시크릿, 계정, 역할을 확인해야 합니다. 특히 `friend-agent-runtime`의 권한은 제거하면 안 됩니다.

이 단계의 핵심은 새 계정으로 전환한 뒤 정상 작동을 검증하고, 그다음 기존 권한을 회수한다는 순서입니다.

8. 객관식 퀴즈 정답

문제: 자동 배포 후 `/version`의 `commit` 값이 GitHub 저장소의 최신 커밋 SHA와 다릅니다. 가장 먼저 확인할 곳은 어디일까요?

정답은 2번입니다.

GitHub Actions 탭에서 최신 커밋으로 시작된 실행이 성공했는지 확인하고, Cloud Run 리비전 탭에서 해당 리비전이 실제로 트래픽을 받는지 살펴봐야 합니다.

API 키 자체나 로컬 `.env` 파일, 캐릭터 설정을 먼저 의심하기보다 어떤 커밋이 배포되었고 실제로 어느 리비전이 서비스 중인지부터 확인하는 것이 논리적인 순서입니다.

9. AI 개발자와 1인 AI 사업에 왜 중요할까요?

이번 강의는 단순히 배포를 자동화하는 데 그치지 않습니다. AI 서비스를 운영할 때 필요한 세 가지 역량을 함께 익히는 과정입니다.

1. 반복 작업 자동화: 코드를 `main`에 머지하면 정해진 절차에 따라 배포할 수 있습니다.
2. 운영 상태 검증: 배포 성공 메시지만 믿지 않고 리비전, 트래픽, 서비스 계정, 커밋 SHA를 확인합니다.
3. 보안 권한 관리: 새 계정이 정상적으로 작동하는 것을 검증한 후 불필요한 권한을 회수합니다.

앞으로 MSM이나 다른 AI 에이전트를 클라우드에 올릴 때도 같은 원리를 적용할 수 있습니다. 기능을 만드는 것뿐 아니라 안전하게 배포하고, 올바른 버전이 실행되는지 확인하며, 권한을 최소화하는 것까지 서비스 개발의 일부로 생각해야 합니다.

10. 핵심 요약 및 반드시 기억할 내용

- 워크플로 파일은 `.github/workflows/`에 있어야 GitHub Actions가 실행합니다.
- 템플릿 파일은 임의로 수정하지 않고, 브랜치와 PR을 통해 검토한 뒤 `main`에 반영합니다.
- PR 머지는 자동 배포의 시작점이 될 수 있으므로, 머지 전에 워크플로 설정을 확인합니다.
- GitHub Actions의 성공뿐 아니라 Cloud Run 리비전, 트래픽, 서비스 계정, `/version`의 커밋 SHA도 확인합니다.
- 새 런타임 계정의 정상 작동을 검증한 뒤 기본 서비스 계정의 불필요한 시크릿 읽기 권한을 회수합니다.
- AI 에이전트가 권한을 임의로 확대하지 않도록 명령 실행 전 검토와 승인을 유지합니다.

한 문장 요약: 검토된 워크플로를 `main`에 반영해 자동 배포하고, 실제 실행 버전과 계정 권한을 검증한 뒤 불필요한 기존 권한까지 회수하는 것이 이번 챕터의 핵심입니다.

Chapter06

/강의정리 챕터 06. 머지만 하면 나간다는 것

1. 학습 목적

이번 강의의 핵심은 자동 배포는 코드를 빠르게 사용자에게 전달하지만, 그 코드가 올바른지까지 보장하지는 않는다는 점을 이해하는 것입니다.

앞선 챕터에서는 GitHub Actions를 설정하고, `main`에 변경 사항이 들어오면 Cloud Run에 자동 배포되도록 만들었습니다. 이번에는 의도적으로 잘못된 코드 두 가지를 배포해 보고, 서비스가 시작하지 못하는 오류와 서비스는 정상적으로 시작하지만 동작이 틀리는 오류의 차이를 경험합니다.

이번 강의에서 꼭 이해해야 할 것은 다음 네 가지입니다.

- 자동 배포가 성공했다는 것과 서비스가 올바르게 동작한다는 것은 다릅니다.
- 새 버전이 시작하지 못하면 Cloud Run이 기존 정상 버전을 유지할 수 있습니다.
- 서비스가 시작되더라도 잘못된 결과를 내면 그 오류는 사용자에게 전달될 수 있습니다.
- 문제가 발생하면 `main`을 직접 덮어쓰지 않고 Revert PR로 되돌립니다.

2. 중학생도 이해할 수 있는 설명

자동 배포를 자동문에 비유해 보겠습니다.

자동문을 설치하면 사람이 매번 문을 열지 않아도 방문객이 들어올 수 있습니다. 하지만 문이 자동으로 열린다고 해서 문이 제대로 설치되었다는 뜻은 아닙니다.

- 문이 아예 열리지 않으면 방문객은 들어갈 수 없습니다.
- 문은 열리지만 엉뚱한 방향으로 열리면 방문객은 잘못된 곳으로 들어갈 수 있습니다.

자동 배포도 같습니다. `main`에 변경 사항이 머지되면 배포가 자동으로 진행되지만, 배포 자동화 자체가 코드의 모든 오류를 찾아내지는 못합니다.

이번 강의에서는 두 가지 상황을 비교합니다.

3. 첫 번째 사고: 앱이 아예 시작하지 않는 변경

어떤 변경을 했나요?

첫 번째 패치 `rename-key.patch`는 LLM API 키를 읽는 환경변수 이름을 다음과 같이 바꿉니다.

- 기존: `LLM_API_KEY`
- 변경: `OPENAI_API_KEY`

코드만 보면 이름을 정리한 것처럼 보입니다. 로컬 `.env` 파일의 이름까지 같이 바꾸면 로컬에서는 정상적으로 작동할 수 있습니다.

하지만 Cloud Run에는 기존 이름인 `LLM_API_KEY`로 시크릿이 연결되어 있습니다. 코드가 `OPENAI_API_KEY`를 찾으면 키를 찾지 못해 앱이 시작하자마자 종료됩니다.

어떻게 확인하나요?

```
git switch main
git pull
git switch -c rename-key

git apply starter/incidents/rename-key.patch
git commit -am "LLM API 키 환경변수 이름을 OPENAI_API_KEY 로 변경"
git push -u origin rename-key

gh pr create --base main \
  --title "LLM API 키 환경변수 이름 변경" \
  --body "starter/incidents/rename-key.patch"

gh pr merge --merge --delete-branch
gh run watch
```

배포 실행은 실패합니다. 하지만 여기서 중요한 점은 사용자가 반드시 오류를 경험하는 것은 아니라는 것입니다.

Cloud Run은 새 리비전이 시작 준비 검사를 통과해야 트래픽을 전환합니다. 새 리비전이 시작하지 못하면 트래픽을 받지 못하고, 기존 정상 리비전이 계속 요청을 처리할 수 있습니다.

따라서 다음 두 상황이 동시에 발생할 수 있습니다.

- GitHub Actions: 새 버전 배포 실패
- 사용자 화면: 이전 버전이 계속 정상적으로 응답

이것은 새 코드가 정상이라는 의미가 아닙니다. `main`에는 여전히 잘못된 코드가 들어 있으므로 다른 변경을 추가로 머지하기 전에 문제를 되돌려야 합니다.

어떻게 되돌리나요?

GitHub 저장소에서 다음 순서로 진행합니다.

1. `Pull requests → Closed`에서 `LLM API 키 환경변수 이름 변경` PR을 엽니다.
2. PR 아래쪽의 `Revert` 버튼을 누릅니다.
3. 새로 생성된 Revert PR에서 `OPENAI_API_KEY`가 `LLM_API_KEY`로 돌아가는지 확인합니다.
4. Revert PR을 머지합니다.
5. 새 자동 배포가 성공하는지 확인합니다.

Revert는 기존 커밋 기록을 지우는 방식이 아닙니다. 기존 변경을 반대로 적용하는 새로운 변경을 만들어 기록을 유지합니다.

핵심: 앱이 시작하지 못하는 오류는 배포 단계나 시작 준비 검사에서 드러날 수 있으며, Cloud Run이 기존 정상 리비전을 계속 제공할 수 있습니다.

4. 두 번째 사고: 앱은 시작하지만 동작이 틀리는 변경

첫 번째 오류와 두 번째 오류의 차이가 이번 강의에서 가장 중요합니다.

어떤 변경을 했나요?

두 번째 패치 `reminder-timezone.patch`는 약속을 저장하기 전에 시각에서 시간대 정보를 제거합니다.

코드에는 저장 형식을 단순하게 만든다는 주석이 달려 있어 리뷰에서 쉽게 지나칠 수 있습니다.

이 변경을 머지하고 배포한 뒤 다음과 같이 요청해 봅니다.

> 내일 아침 8시에 물 마시라고 알려 줘.

캐릭터의 답변에는 아침 8시라고 표시되지만, 약속 안내와 기억·약속 화면에는 다음 날 오후 5시인 `17:00`이 저장됩니다.

왜 9시간 차이가 발생할까요?

한국 시간과 세계 표준시(UTC)는 9시간 차이가 납니다.

상황	시간
사용자가 요청한 한국 시간	08:00
시간대 정보가 제거된 뒤 UTC 기준으로 해석된 시간	08:00 UTC
화면에서 한국 시간으로 변환한 결과	17:00 KST

시간대 정보가 사라진 시각은 실행 환경의 시간대에 따라 다르게 해석될 수 있습니다. 로컬 환경은 한국 시간으로 동작하지만 Cloud Run 서버는 UTC를 사용하므로 같은 코드가 서로 다른 결과를 낼 수 있습니다.

왜 배포는 성공했을까요?

이번에는 앱이 정상적으로 시작하고 새 리비전도 트래픽을 받습니다. 따라서 배포 시스템 입장에서는 작업이 성공한 것입니다.

하지만 실제 약속 시간은 틀렸습니다.

즉, 기술적으로 실행 가능한 코드와 업무적으로 올바른 코드는 서로 다를 수 있습니다.

이 오류를 알아내려면 단순히 배포 성공 여부만 확인해서는 부족합니다. 실제 사용자 요청을 보내고, 저장된 결과가 기대한 값과 일치하는지 확인해야 합니다.

어떻게 되돌리나요?

첫 번째 사고와 같은 방식으로 Revert PR을 만들어 되돌립니다.

1. `약속 시각 저장 형식 단순화` PR을 찾습니다.
2. 해당 변경을 반대로 되돌리는 PR을 만듭니다.
3. 바뀐 줄을 확인한 뒤 머지합니다.
4. 자동 배포가 성공하는지 확인합니다.
5. 같은 요청을 다시 보내 약속 시간이 `08:00`으로 저장되는지 확인합니다.

이번 실습에서는 약속이 인스턴스 메모리에만 저장되어 새 리비전이 뜨면 잘못된 약속도 사라집니다. 하지만 데이터베이스에 이미 잘못된 값이 저장되는 서비스라면 코드를 되돌리는 것만으로 데이터까지 복구되지는 않습니다. 잘못 저장된 데이터는 별도로 찾아 수정해야 합니다.

5. 두 사고의 차이 비교

구분	첫 번째: 환경변수 이름 오류	두 번째: 시간대 오류
변경 내용	API 키 환경변수 이름 변경	약속 시각에서 시간대 정보 제거
Actions 결과	배포 실패	배포 성공
앱 시작	시작하지 못함	정상 시작
사용자 영향	기존 리비전이 계속 응답할 수 있음	잘못된 약속 시간이 사용자에게 노출됨
원인 확인	Cloud Run 로그의 시작 오류	실제 요청 결과와 약속 저장 화면
복구 방법	Revert PR	Revert PR 및 결과 재검증
핵심 교훈	시작 실패는 배포 과정에서 발견될 수 있음	정상 실행만으로 업무 정확성을 보장할 수 없음

이 표에서 가장 중요한 차이는 첫 번째 오류는 실행 자체를 막았지만, 두 번째 오류는 실행을 막지 않았다는 점입니다.

자동 배포가 확인할 수 있는 범위는 워크플로와 검사 설정에 따라 다릅니다. 앱이 시작하는지 확인하는 검사만으로는 잘못된 약속 시간이 저장되는 문제까지 알아내기 어렵습니다.

6. 머지 전에 두 번째 오류를 발견하려면?

PR에서 변경된 줄을 읽는 것만으로는 부족할 수 있습니다. 시간대 정보가 사라져도 코드 자체는 문법적으로 올바르고 정상적으로 실행될 수 있기 때문입니다.

이런 오류를 줄이려면 다음과 같은 검증이 필요합니다.

- 코드 리뷰: 시각을 저장하고 변환하는 과정에서 시간대 정보가 유지되는지 검토합니다.
- 자동 테스트: 한국 시간으로 `08:00`을 입력했을 때 저장 결과도 의도한 시간으로 유지되는지 테스트합니다.
- 실행 환경 테스트: 로컬뿐 아니라 Cloud Run과 같은 UTC 환경에서도 같은 결과가 나오는지 확인합니다.
- 배포 후 기능 검증: 실제 요청을 보내고 응답뿐 아니라 저장된 결과까지 확인합니다.

이것은 이번 강의의 사례를 바탕으로 도출할 수 있는 개선 방향입니다. 강의에서 직접 수행한 핵심 실습은 두 변경을 머지하고 결과를 관찰한 다음 Revert PR로 복구하는 것입니다.

7. AI 개발자에게 왜 중요할까요?

AI 코딩 에이전트는 코드를 빠르게 변경하고 PR을 만들 수 있습니다. 하지만 변경 내용이 실제 운영 환경에서 어떤 결과를 낼지까지 자동으로 보장하는 것은 아닙니다.

이번 사례는 AI를 활용해 서비스를 만드는 사람에게 세 가지 원칙을 알려 줍니다.

1. 코드가 실행되는 환경을 고려해야 합니다. 로컬에서는 작동해도 클라우드에서는 환경변수, 시간대, 권한 등이 달라질 수 있습니다.
2. 자동화의 성공 조건을 구분해야 합니다. 배포 성공은 코드가 배포되었다는 의미이지, 사용자 요구를 올바르게 처리했다는 의미가 아닙니다.
3. 복구 절차를 준비해야 합니다. 오류가 발생하면 `main`을 직접 수정하기보다 Revert PR로 변경 기록을 남기고 안전하게 되돌립니다.

MSM 같은 AI 서비스에도 이 원칙을 적용할 수 있습니다. 예를 들어 AI가 사용자의 글을 분석해 스토리를 생성하는 기능을 배포했다면, 서비스가 시작되는지만 확인할 것이 아니라 원문 보존, 문체 유지, 생성 결과의 형식 등 핵심 기능이 실제로 제대로 작동하는지도 확인해야 합니다.

8. 주관식 퀴즈 모범 답안

문제: 두 변경 모두 PR 화면에서 바뀐 줄을 보고 머지했는데, 첫 번째 변경은 사용자에게 나가지 않았고 두 번째 변경은 나갔습니다. 이유는 무엇이며, 두 번째 변경을 머지 전에 발견하려면 어떤 확인이 필요했을까요?

모범 답안

첫 번째 변경은 코드가 `OPENAI_API_KEY`를 찾지만 Cloud Run에는 `LLM_API_KEY`라는 이름으로 시크릿이 연결되어 있어 앱이 시작하지 못했습니다. Cloud Run의 시작 준비 검사를 통과하지 못해 새 리비전이 트래픽을 받지 못했고, 기존 정상 리비전이 계속 요청을 처리했습니다.

반면 두 번째 변경은 앱 자체는 정상적으로 시작했지만 시간대 정보를 제거하면서 한국 시간과 UTC 사이의 차이가 잘못 처리되었습니다. 배포 시스템은 앱이 실행되는 것을 확인했지만 약속 시간이 올바른지까지 확인하지 못해 오류가 사용자에게 전달되었습니다.

두 번째 변경을 머지 전에 발견하려면 코드 리뷰에서 시간대 처리 방식을 확인하고, 한국 시간 `08:00`을 입력했을 때 저장 결과도 의도한 시각인지 검증하는 자동 테스트를 추가했어야 합니다. 실제 클라우드 환경에서도 기능 테스트를 수행하면 오류를 더 일찍 발견할 수 있습니다.

9. 반드시 기억할 핵심

- `main`에 머지된 변경은 자동 배포를 통해 사용자에게 빠르게 전달될 수 있습니다.
- 새 리비전이 시작하지 못하면 기존 정상 리비전이 계속 트래픽을 받을 수 있습니다.
- 앱이 정상적으로 시작하더라도 기능 결과가 틀리면 오류가 사용자에게 전달됩니다.
- 배포 성공 여부와 기능 정확성은 별도로 검증해야 합니다.
- 오류가 발생하면 Revert PR로 변경을 되돌리고, 복구 배포와 기능 결과까지 확인합니다.
- 데이터베이스에 이미 저장된 잘못된 데이터는 코드 복구만으로 해결되지 않을 수 있습니다.

한 문장 요약: 자동 배포는 변경을 빠르게 전달할 뿐 정확성을 보장하지 않으므로, 실행 여부와 기능 결과를 각각 검증하고 Revert PR을 통해 안전하게 복구해야 합니다.

Chapter07

/강의정리 챕터 07. 과제 고르기

1. 오늘 수업의 목적

오늘의 핵심은 자동 배포를 실제 기능 추가에 적용하고, 보안 권한이 의도대로 작동하는지 확인하는 것입니다.

앞선 챕터에서는 GitHub에서 코드를 `main` 브랜치에 머지하면 GitHub Actions가 Google Cloud에 자동 배포되도록 만들었습니다. 이번에는 그 구조를 실제로 사용하면서 세 가지를 확인합니다.

- 자동화: 사람이 배포 명령을 직접 실행하지 않아도 기능이 서비스에 반영되는가?
- 보안: 허용되지 않은 저장소에서 배포하려고 하면 Google Cloud가 차단하는가?

- 추적: 배포를 누가, 어느 저장소에서 실행했는지 기록으로 확인할 수 있는가?

비유하면, 지금까지는 자동문을 설치한 단계였다면 오늘은 자동문이 실제로 열리는지, 허가받지 않은 사람이 들어오려 하면 막히는지, 누가 출입했는지 기록이 남는지를 검사하는 단계입니다.

2. 네 가지 과제 이해하기

과제 1. 기능 추가 및 자동 배포

기본 · 수업 중

작은 기능을 만들고 PR을 통해 머지한 다음, 자동 배포와 실제 화면 반영을 확인합니다.

핵심 질문: 코드를 머지한 뒤 사람이 배포하지 않아도 기능이 바뀌는가?

과제 2. 다른 저장소의 배포 차단

다른 GitHub 저장소에서 같은 워크플로를 실행해 인증이 거부되는지 확인합니다.

핵심 질문: 내 저장소가 아닌 곳에서도 Google Cloud에 배포할 수 있는가?

과제 3. 감사 로그로 배포 추적

Google Cloud 로그에서 배포 서비스 계정과 GitHub 저장소·브랜치에 관한 신원 정보를 찾습니다.

핵심 질문: 문제가 생겼을 때 어떤 주체가 배포를 시작했는지 추적할 수 있는가?

과제 4. 다른 CD 방식 조사

Cloud Build, Cloud Deploy, GitLab CI/CD, Argo CD 등과 오늘의 방식을 비교합니다.

핵심 질문: 서비스의 규모와 운영 방식에 따라 어떤 배포 구조가 적합한가?

3. 과제 1을 단계별로 이해하기

이번 수업에서 우선 수행할 과제입니다. 처음부터 복잡한 기능을 만들 필요는 없습니다. 작고, 눈으로 확인할 수 있고, 기존 기능에 영향을 적게 주는 변경을 선택하는 것이 좋습니다.

1단계. 무엇을 바꿀지 결정하기

예를 들어 다음 기능 중 하나를 선택할 수 있습니다.

- 화면 아래에 현재 배포된 커밋의 앞 7자리를 표시합니다.
- 입력창 위에 사용자가 참고할 추천 문장을 몇 개 추가합니다.
- 캐릭터 만들기 화면에 새로운 말투 선택지를 추가합니다.

초보자에게는 기존 기능의 작동 방식을 크게 바꾸지 않는 추천 문장 추가가 무난합니다. 이미 `/version` 응답을 읽어 화면에 표시하는 기능이 있다면 커밋 SHA 표시도 좋은 선택입니다.

2단계. 코딩 에이전트에게 변경을 맡기기

코딩 에이전트에게는 변경 목적, 화면 위치, 유지해야 할 기존 기능을 명확하게 알려야 합니다.

예를 들어 다음과 같이 요청할 수 있습니다.

현재 애플리케이션에 작은 화면 기능 하나를 추가해 주세요.

목표: 사용자가 입력창에 무엇을 입력할지 쉽게 이해하도록 추천 문장 3개를 표시합니다.

요구사항

1. 추천 문장은 입력창 바로 위에 표시합니다.
2. 문장은 짧고 이해하기 쉬운 한국어로 작성합니다.
3. 추천 문장을 클릭하면 해당 문장이 입력창에 들어가도록 합니다.
4. 기존 기능과 화면 디자인은 가능한 한 유지합니다.
5. 기존 환경 변수, 시크릿 이름, 배포 설정은 변경하지 않습니다.
6. 변경 내용을 테스트하고, 수정한 파일과 테스트 결과를 설명해 주세요.

작업은 별도의 Git 브랜치에서 진행하고, `main` 브랜치에 직접 커밋하지 마세요.

3단계. PR을 통해 머지하기

작업 브랜치에서 기능 개발

코딩 에이전트가 코드 수정 및 테스트

커밋 및 Pull Request 생성

변경 내용을 검토하고 확인

main 브랜치에 머지

GitHub Actions 자동 실행

Google Cloud 배포 및 검증

리비전 · 커밋 SHA · 실제 화면 확인

이 과정에서 중요한 점은 머지 자체가 배포 명령을 대신하는 것이 아니라, 머지가 자동 배포 워크플로를 시작하는 계기가 된다는 것입니다.

4단계. 배포가 제대로 되었는지 확인하기

코드를 머지했다고 해서 과제가 끝난 것은 아닙니다. 다음 네 가지를 확인해야 합니다.

확인 위치	확인할 내용
GitHub Actions	머지 커밋으로 시작된 `deploy` 실행이 성공했는가?
Cloud Run 리비전	새 리비전이 트래픽 100%를 받고 있는가?
서비스 URL + `/version`	응답의 `commit` 값이 머지한 커밋 SHA와 일치하는가?
실제 서비스 화면	추가한 기능이 사용자 화면에 표시되는가?

브라우저에 이전 화면이 남아 있으면 Windows에서는 `Ctrl + F5`로 강력 새로고침을 해 보세요.

완료 기준: 자동 배포 성공, 새 리비전의 트래픽 전환, 커밋 SHA 일치, 기능의 실제 표시까지 확인해야 합니다.

4. 나머지 과제는 무엇을 배우는가?

과제 2. 다른 저장소의 배포 시도

앞선 챕터에서 설정한 WIF(Workload Identity Federation)의 보안 효과를 검증합니다.

기대하는 결과

내 저장소에서는 인증에 성공하지만, 다른 저장소에서는 인증 단계에서 거부됩니다.

확인할 점

실패 지점이 `google-github-actions/auth` 단계인지, Cloud Run 리비전이 변경되지 않았는지 확인합니다.

중학생 수준으로 비유하면, 건물에 출입증이 있어도 되는 것이 아니라 출입증에 등록된 소속까지 확인하는 보안 시스템입니다.

실습용 저장소를 만들 때는 실제 배포 권한과 시크릿을 불필요하게 복제하지 말고, 과제에 필요한 범위만 구성하세요. 실습이 끝난 뒤 저장소를 삭제하려면 이름을 다시 확인하고, 필요한 기록은 먼저 보관합니다.

과제 3. 감사 로그로 배포 추적

자동 배포가 성공했을 때 Google Cloud에 어떤 신원으로 기록되는지 확인합니다.

수업 자료에서 안내한 순서는 다음과 같습니다.

1. Google Cloud 콘솔에서 로깅 → 로그 탐색기를 엽니다.
2. 아래 조건으로 Cloud Run 관리 활동 기록을 검색합니다.

```
protoPayload.serviceName="run.googleapis.com"
logName:"cloudaudit.googleapis.com%2Factivity"
```

1. 가장 최근 배포 시각의 기록을 펼칩니다.
2. `authenticationInfo`의 `principalEmail`에서 `github-deployer@...` 계정인지 확인합니다.
3. `serviceAccountDelegationInfo`에서 GitHub 저장소 이름과 `refs/heads/main`에 관한 신원 정보가 있는지 살펴봅니다.

자료에서 기대하는 결과는 사람이 직접 배포했을 때는 사람의 계정이, GitHub Actions가 배포했을 때는 배포 서비스 계정과 위임된 신원 정보가 기록되는 것입니다. 다만 로그의 실제 필드와 표시 형식은 설정과 인증 방식에 따라 달라질 수 있습니다.

과제 4. 다른 CD 방식 비교

CD(Continuous Delivery/Deployment)는 변경된 코드를 배포 가능한 상태로 만들거나 실제 서비스에 자동 반영하는 과정입니다.

방식	핵심 특징
GitHub Actions + WIF	GitHub 워크플로에서 작업하고, OIDC 기반으로 Google Cloud 권한을 얻습니다.
Cloud Build	Google Cloud의 빌드 서비스를 이용해 빌드와 배포를 구성할 수 있습니다.
Cloud Deploy	배포 단계, 승인, 단계별 출시를 관리하는 데 초점을 둡니다.
GitLab CI/CD	GitLab 저장소의 파이프라인으로 테스트와 배포를 자동화합니다.
Argo CD	Git 저장소에 선언된 상태를 Kubernetes 환경이 따라가도록 하는 GitOps 도구입니다.
Vercel	지원하는 웹 프레임워크와 저장소를 연결해 웹 애플리케이션 배포를 간편하게 할 수 있습니다.

위 표는 개념적인 비교입니다. 구체적인 비용, 승인 기능, Google Cloud 인증 구성은 각 제품의 요금제와 설정에 따라 달라집니다.

5. 왜 AI 서비스 개발자에게 중요한가?

AI 에이전트는 일반적인 웹 애플리케이션보다도 예상하지 못한 결과가 발생할 가능성이 있습니다. 프롬프트나 도구 호출 로직을 바꾸는 것만으로도 서비스 결과가 달라질 수 있기 때문입니다.

따라서 배포를 자동화하는 것과 배포 결과를 검증하는 것은 별개의 일입니다.

- Git 브랜치와 PR: 변경 이력을 관리하고 검토합니다.

- 자동 테스트: 알려진 오류를 배포 전에 발견합니다.
- WIF와 최소 권한: 허가되지 않은 주체의 배포 접근을 제한합니다.
- Cloud Run 리비전: 문제가 발생했을 때 어떤 버전이 실행되었는지 확인합니다.
- 감사 로그: 배포 주체와 관련 신원 정보를 추적합니다.
- 기능 테스트: 서비스가 실행된다는 사실을 넘어 실제 결과가 올바른지 확인합니다.

특히 챕터 06에서 배운 것처럼, 서비스가 정상적으로 시작되었다고 해서 기능까지 정확한 것은 아닙니다.

6. 현재 학습 및 창업 목표와 연결하기

1인 AI 사업을 준비하시는 입장에서는 이번 과제가 단순한 배포 실습을 넘어 혼자서 AI 서비스를 안전하게 운영하는 기본 역량을 익히는 과정입니다.

예를 들어 My-Story-Maker(MSM)에서 글을 분석하고 웹소설 형태로 변환하는 기능을 추가한다고 생각해 보겠습니다.

1. 새 브랜치에서 글 분석 결과에 대한 설명을 추가합니다.
2. PR에서 기존 글의 원문이 훼손되지 않았는지, 테스트 결과가 적절한지 검토합니다.
3. 머지 후 자동 배포를 진행합니다.
4. 실제 서비스에서 글 분석 결과를 확인합니다.
5. 문제가 발생하면 리비전과 감사 로그를 확인하고, 필요한 경우 변경을 되돌립니다.

이런 구조가 자리 잡으면 기능을 수정할 때마다 수동 배포 절차를 반복하는 부담을 줄이면서, 변경 내용을 추적하고 관리할 수 있습니다.

7. 반드시 기억할 내용

- 자동 배포는 머지 이후의 배포 과정을 자동화하는 것이지, 기능의 정확성을 보장하는 것은 아닙니다.
- WIF의 저장소 조건은 허용된 저장소와 신원을 제한하는 보안 장치입니다.
- Cloud Run 리비전의 배포 상태와 실제 기능 동작을 모두 확인해야 합니다.
- 감사 로그는 배포가 어떤 신원으로 수행되었는지 추적하는 데 도움이 됩니다.
- 문제가 생겼을 때는 원인을 확인하고, PR을 통한 되돌리기와 검증 절차를 활용합니다.

8. 한 문장 요약

오늘의 핵심은 코드를 자동으로 배포하는 데서 끝나는 것이 아니라, 배포가 정확하게 이루어지고 허가된 주체만 실행하며 문제가 생겼을 때 원인을 추적할 수 있는지까지 검증하는 것입니다.

제안드리는 실습 순서는 과제 1을 먼저 완료한 뒤, 과제 3으로 배포 기록을 확인하고, 과제 2로 보안 차단을 검증한 다음, 과제 4에서 대안을 비교하는 순서입니다. 수업에서 요구한 과제 1을 우선하되, 나머지 과제는 시간과 환경에 맞춰 진행하시면 됩니다.

Chapter08

/강의정리 챕터 08. 자동 배포와 권한 설정 회고

1. 학습 목적

이번 수업에서는 자동 배포가 어떤 과정을 거쳐 실행되는지 직접 확인하고, 그 과정에서 코드를 실행하는 주체와 Google Cloud에 접근할 수 있는 권한을 어떻게 통제하는지 배웠습니다.

주관식 퀴즈의 핵심은 처음 Vercel을 사용했을 때 추측했던 내용을 오늘 직접 만든 GitHub Actions와 WIF 설정에 비추어 돌아보는 것입니다.

2. 중학생도 이해할 수 있는 설명

자동 배포를 식당에 비유해 보겠습니다.

- GitHub 저장소: 요리법과 재료가 보관된 주방
- 워크플로 파일: 요리 순서가 적힌 조리 지침서
- 러너(Runner): 지침서를 받아 실제 작업하는 임시 조리 공간
- 권한 설정: 주방에 들어올 수 있는 사람과 사용할 수 있는 도구를 제한하는 출입 통제
- Cloud Run: 완성된 요리를 손님에게 제공하는 식당
- `/version`: 어떤 조리법 버전으로 요리를 만들었는지 확인하는 표식

GitHub Actions의 러너는 작업이 실행될 때 새 작업 환경에서 시작하고, 저장소의 파일을 내려받아 지침에 따라 작업합니다. 단계 하나가 실패하면 기본적으로 뒤의 단계들은 실행되지 않습니다.

또한 작업을 실행할 수 있다는 것과 Google Cloud에 배포할 권한이 있다는 것은 별개입니다. WIF는 GitHub에서 발급된 OIDC 토큰을 확인해, 허용된 저장소의 작업이 지정된 서비스 계정을 사용할 수 있도록 합니다.

3. 오늘 배운 핵심 내용

① 자동 배포 구조

워크플로 파일은 다음 요소를 정의합니다.

요소	역할
`on`	어떤 이벤트가 발생했을 때 실행할지 결정
`runs-on`	어떤 러너 환경에서 실행할지 결정
`steps`	어떤 작업을 어떤 순서로 실행할지 결정
`permissions`	워크플로가 사용할 수 있는 권한을 제한

실제 배포 흐름은 다음과 같습니다.

코드 수정 및 PR

main 브랜치에 머지

GitHub Actions 실행

러너가 코드와 워크플로를 받아 작업

Google Cloud 인증 및 배포

Cloud Run 리비전 및 `/version` 확인

② 권한을 나누는 방식

오늘의 보안 설정에서 중요한 원칙은 키 파일을 보관하는 대신 필요한 신원을 확인하고, 역할에 따라 권한을 분리하는 것입니다.

- WIF: 허용된 GitHub 저장소의 OIDC 토큰을 확인합니다.
- 배포 서비스 계정: 애플리케이션을 Google Cloud에 배포하는 데 필요한 권한을 갖습니다.
- 런타임 서비스 계정: 실행 중인 애플리케이션에 필요한 권한만 갖습니다.
- 최소 권한 원칙: 서비스가 실제로 필요한 권한만 부여합니다.

이렇게 나누면 애플리케이션이 실행될 때 사용하는 권한과 새 버전을 배포할 때 사용하는 권한을 구분할 수 있습니다.

③ 실패에서 배운 것

변경 유형	결과	배운 점
앱이 시작되지 않는 변경	새 리비전이 정상적으로 시작되지 않아 기존 리비전이 요청을 계속 처리할 수 있음	배포 실패가 사용자 서비스 중단으로 반드시 이어지는 것은 아님
앱은 시작되지만 기능이 잘못된 변경	배포 자체는 성공해도 잘못된 결과가 사용자에게 전달될 수 있음	배포 성공과 기능 정확성은 다름

두 번째 유형의 오류는 코드 검토, 기능 테스트, 실제 배포 환경에서의 확인이 중요하다는 것을 보여 줍니다.

또한 이미 `main`에 머지된 잘못된 코드는 원래의 커밋을 지우는 대신 Revert PR로 반대 변경을 추가해 되돌릴 수 있습니다.

4. 주관식 퀴즈 모범 답안

Q. 섹션 1에서 Vercel이 푸시 뒤 몇 분 동안 무엇을 했는지, 어떤 권한으로 내 코드를 가져갔는지 짐작해 적었습니다. 그 답을 다시 읽고, 오늘 직접 만든 워크플로와 권한 설정에 비춰 짐작이 맞았던 부분과 새로 알게 된 부분을 적어 보세요.

처음 Vercel을 사용했을 때는 GitHub에 코드를 푸시하면 Vercel이 코드를 가져와 빌드하고, 필요한 설정을 적용한 뒤 웹 서비스에 배포한다고 추측했습니다. 푸시 이후 몇 분 동안 자동으로 여러 작업이 진행된다는 점은 맞게 이해했습니다.

오늘 GitHub Actions 워크플로를 직접 만들면서, 자동 배포는 단순히 코드를 가져가는 과정이 아니라 실행 조건(`on`), 작업 환경(`runs-on`), 단계(`steps`), 권한(`permissions`)을 정해진 순서대로 수행하는 과정이라는 것을 알게 되었습니다. 또한 러너는 실행할 때마다 새로운 환경에서 시작하며, 저장소에 커밋된 파일을 내려받아 작업하고, 단계 하나가 실패하면 이후 단계가 실행되지 않을 수 있다는 점도 배웠습니다.

권한에 대해서는 처음에는 Vercel이 GitHub에 접근할 수 있는 권한을 받아 코드를 가져오는 정도로 생각했습니다. 하지만 오늘은 GitHub Actions가 Google Cloud에 배포할 때 WIF를 사용해 OIDC 토큰을 확인하고, 허용된 저장소의 작업만 배포 서비스 계정을 사용할 수 있도록 설정했습니다. 배포 서비스 계정과 애플리케이션 실행용 서비스 계정도 분리하고 각각 필요한 권한만 부여했습니다.

새롭게 알게 된 가장 중요한 사실은 자동 배포가 성공했다고 해서 기능까지 정확하다는 보장은 없다는 것입니다. 애플리케이션이 시작되지 않는 오류는 배포 과정에서 발견될 수 있지만, 애플리케이션은 정상적으로 실행되면서 기능만 잘못된 경우에는 별도의 테스트가 필요합니다.

결국 처음의 추측은 자동 배포의 큰 흐름에서는 맞았지만, 실제로 어떤 단계가 실행되고 어떤 권한으로 클라우드에 접근하며 결과를 어떻게 검증하는지까지는 알지 못했습니다. 앞으

로는 자동 배포를 단순한 편의 기능이 아니라, 실행 절차와 권한을 통제하고 결과를 검증할 수 있는 운영 체계로 이해하겠습니다.

5. 왜 AI 서비스 개발자에게 중요한가?

AI 서비스를 만들 때 프롬프트, 모델 호출, 도구 사용 로직을 수정하면 예상하지 못한 결과가 생길 수 있습니다. 따라서 자동 배포 기능만 구현하는 것이 아니라, 변경이 안전하게 전달되고 문제가 생겼을 때 추적·복구할 수 있는 구조를 마련해야 합니다.

특히 1인 AI 사업을 운영할 때는 모든 배포를 직접 수동으로 처리하기 어렵기 때문에 다음 세 가지를 함께 갖추는 것이 중요합니다.

1. 자동화: 변경 사항을 일정한 절차로 배포합니다.
2. 보안: 배포 권한과 서비스 실행 권한을 분리합니다.
3. 검증: 로그, 리비전, 커밋 SHA, 실제 기능을 확인합니다.

6. 반드시 기억할 내용

- 자동 배포는 정해진 이벤트에 따라 워크플로를 실행하는 구조입니다.
- GitHub Actions의 실행 권한과 Google Cloud 배포 권한은 별개입니다.
- WIF는 장기 보관하는 키 파일 없이 신원을 확인할 수 있도록 해 줍니다.
- 배포가 성공했는지와 서비스 기능이 올바른지는 각각 확인해야 합니다.
- 로그와 커밋 SHA를 활용하면 배포 결과를 추적하는 데 도움이 됩니다.

한 문장 요약: 오늘의 학습은 자동 배포를 만드는 방법뿐 아니라, 누가 배포할 수 있는지 통제하고 배포 결과가 실제로 올바른지 확인하는 방법까지 이해한 것입니다.